



《AI大模型Ragent项目》——流式路由的首包探测机制

来自： 拿个offer-开源&项目实战



马丁

2026年04月13日 09:37

上一篇拆解了供应商级别的流式调用实现——doStreamChat 如何通过 StreamAsyncExecutor 异步提交、OpenAIStyleSseParser 逐行解析 SSE、StreamCallback 回调推送、StreamCancellationHandle 取消机制，把一次流式调用从请求构建到数据推送全链路跑通。但那些都是一个 ChatClient 连接一个供应商的实现。在路由层面，还有一个更大的问题没解决。之前讲的 executeWithFallback——同步调用可以逐个尝试候选，调成功了返回结果，调失败了 catch 住切换下一个。但流式调用的 client.streamChat() 调用后立即返回取消句柄，真正的数据在异步线程通过回调推送。等你发现第一个供应商不行（onError 被调用了），取消句柄已经返回给调用方了，前端可能已经收到了几个 token——来不及切换了。这一篇解答这个问题：RoutingLLMService.streamChat() 是怎么在流式调用的异步特性下实现故障转移的。

阅读提示：本篇讲的首包探测机制，核心解决的是本地部署模型（Ollama、vLLM 等）在承压时宕机的故障转移问题——HTTP 连接已建立、模型接受了请求，但在生成第一个 token 之前崩溃（GPU 显存溢出、进程崩溃等）。这类问题在企业级本地部署场景中很常见，本地模型直接暴露给应用，没有云平台的负载均衡保护。如果你的项目只使用云 API（百炼、硅基流动等），云端供应商通常自行处理节点故障，请求方看到的要么是快速的 HTTP 错误、要么是正常响应，出现连接成功但首包前崩溃的概率极低。这种情况下首包探测的价值有限，加上整套机制（ProbeStreamBridge + CompletableFuture 同步等待）设计较为复杂，可以选择性查看本篇，或者只看 probe-and-commit 的设计思路，跳过具体实现细节。

流式路由的核心挑战

1. 为什么 executeWithFallback 不够用

第三篇讲的 executeWithFallback 是为同步调用设计的：

```
// 同步路由——一行代码搞定
public String chat(ChatRequest request) {
    return executor.executeWithFallback(
        ModelCapability.CHAT,
        selector.selectChatCandidates(Boolean.TRUE.equals(request.getThinking())),
        target -> clientsByProvider.get(target.candidate().getProvider()),
        (client, target) -> client.chat(request, target) // ← 同步调用，成功返回，失败抛异常
    );
}
```

client.chat(request, target) 是同步的——方法返回时结果已经确定，要么返回正确的 String，要么抛异常。executeWithFallback 的 try-catch 自然能捕获异常并切换到下一个候选。

流式调用就不行了。client.streamChat() 调用后立即返回一个 StreamCancellationHandle，方法返回的那一刻，HTTP 连接可能刚建立，第一个 token 还没到。真正的数据在异步线程上通过 StreamCallback.onContent() 推送，异常通过 StreamCallback.onError() 推送——这些都发生在 streamChat() 方法返回之后，try-catch 捕获不到。

如果硬要用 executeWithFallback 包流式调用，它只能捕获前置校验阶段的同步异常（比如 API Key 缺失），对流式传输过程中的 HTTP 500、网络断开等异步错误无能为力。

1.1 问题一：无法在返回前判断成功失败

executeWithFallback 的前提是：caller.call(client, target) 返回时，调用已经有了明确结果。但流式调用返回的是一个取消句柄，不是调用结果。调用是否成功，要等异步线程开始读取 SSE 数据、收到第一个 token 或错误之后才能确定。

1.2 问题二：数据已推送无法撤回

这个问题更致命。假设直接把真实 callback 传给第一个供应商：

1. 百炼开始流式响应，推了 callback.onContent("AirPods")、callback.onContent(" Pro")
2. 前端已经渲染了 AirPods Pro 三个字
3. 百炼突然报错，callback.onError(ex) 被调用
4. 路由层想切换到硅基流动重试——但那两个 token 已经推给前端了
5. 硅基流动重新生成完整回答，又推了 callback.onContent("AirPods")、callback.onContent(" Pro")
6. 前端看到的是：AirPods Pro（百炼的残片）+ 一个错误 + AirPods Pro 2 的保修期是...（硅基的完整回答）——内容重复且断裂

用户体验完全崩了。核心问题在于：真实 callback 收到的数据无法撤回。一旦 onContent 被调用，前端就渲染了。

1.3 流式路由 ≠ fire-and-forget

看到这里可能有一个疑问：上一篇讲了 `client.streamChat()` 调用后立即返回，请求在异步线程执行。既然是异步的，路由层怎么知道成功还是失败？提交出去不就管不了了吗？

这个疑问混淆了两层 `streamChat` 的行为：

供应商层的 `client.streamChat()`——确实是 fire-and-forget。`StreamAsyncExecutor.submit()` 把任务提交到线程池后立即返回取消句柄，不等结果。上一篇讲的就是这一层

路由层的 `RoutingLLMService.streamChat()`——**在底层异步之上加了一层同步等待**。它调完 `client.streamChat()` 之后，不是直接返回，而是阻塞在 `ProbeStreamBridge.awaitFirstPacket()` 上，等异步线程的首包信号

两层的行为差异用代码说最直接：

```
// 供应商层：提交后立即返回
handle = client.streamChat(request, bridge, target);

// ← 这里 client.streamChat() 已经 return 了，但路由层没有 return

// 路由层：阻塞等待首包信号，最多 60 秒
ProbeStreamBridge.ProbeResult result = awaitFirstPacket(bridge, handle, callback);
// ← 路由线程在这里等着，直到异步线程推了第一个 token 或报了错
```

所以 `RoutingLLMService.streamChat()` 的 for 循环在每个候选上都会同步等待首包结果，拿到结果后才决定 commit 还是切换下一个。它不是提交就不管了，而是用 `CompletableFuture` 把首包这一个关键节点从异步拉回了同步——只等这一个信号，后续 token 推送仍然走异步。

2. probe-and-commit 模式

解决思路是：不要急着把数据推给真实 callback，先探测一下供应商是不是正常的。

具体做法：在真实 callback 和供应商之间插入一个中间层——`ProbeStreamBridge`。它拦截所有回调事件，在确认供应商可用之前先缓冲起来，不转发。

整个流程分三个阶段：

探测阶段——调用 `client.streamChat(request, bridge, target)`，传入的是 `ProbeStreamBridge` 而非真实 callback。异步线程开始流式读取，第一个 token（或错误）到达时通过 `CompletableFuture.complete()` 发出信号。路由线程阻塞在 `bridge.awaitFirstPacket()` 上等待这个信号。

判断阶段——路由线程被唤醒，检查结果。收到了内容？说明供应商正常工作。收到了错误？说明供应商有问题。超时了？说明供应商响应太慢。

提交/切换阶段——如果探测成功，`awaitFirstPacket()` 内部自动调 `commit()`：把缓冲的事件一次性刷给真实 callback，后续事件直通不再缓冲。如果探测失败，取消当前连接，`bridge` 里缓冲的事件直接丢弃（不会转发给真实 callback），然后切换到下一个候选重来。

关键在于：`awaitFirstPacket()` 返回之前，真实 callback 没有收到任何数据。所以切换供应商时前端完全无感知——它既没有收到错误的 token，也没有收到错误通知。

打个比方：你点了一道菜，厨房开始做，但服务员先确认菜没问题才端给你。如果这道菜做坏了，服务员直接倒掉让厨房重做一份，你完全不知道中间废了一份。

RoutingLLMService.streamChat() 完整实现

1. chat() vs streamChat() 的路由策略

先看两者的代码放在一起对比。`chat()` 很简洁：

```
@Override
@RagTraceNode(name = "llm-chat-routing", type = "LLM_ROUTING")
public String chat(ChatRequest request) {
    return executor.executeWithFallback(
        ModelCapability.CHAT,
        selector.selectChatCandidates(Boolean.TRUE.equals(request.getThinking())),
        target -> clientsByProvider.get(target.candidate().getProvider()),
        (client, target) -> client.chat(request, target)
    );
}
```

一行 `executeWithFallback` 搞定——选候选、查客户端、调用、捕获异常、切换，全部由通用执行器处理。

`streamChat()` 自己实现了整套遍历和故障转移逻辑。为什么不能复用 `executeWithFallback`？用一张表格说清楚：

维度	chat()（同步路由）	streamChat()（流式路由）
路由机制	executeWithFallback() 通用执行器	自己实现遍历 + 故障转移
故障检测	try-catch 捕获同步异常	ProbeStreamBridge.awaitFirstPacket() 等待首包结果
结果判断	返回值（成功）或异常（失败）	四种结果：SUCCESS / ERROR / TIMEOUT / NO_CONTENT
数据保护	不需要（同步返回，无中间状态）	ProbeStreamBridge 缓冲事件
切换代价	零（失败直接调下一个）	需要 handle.cancel() 取消当前连接
返回值	String（完整回答）	StreamCancellationHandle（取消句柄）
探测开销	无	每个候选创建独立的 ProbeStreamBridge

不是不想复用 executeWithFallback，而是用不了。它的设计前提是调用是同步的、结果通过返回值或异常传递。流式调用的异步 + 回调模式打破了这个前提。

2. streamChat() 完整代码

```
@Override
@RagTraceNode(name = "llm-stream-routing", type = "LLM_ROUTING")
public StreamCancellationHandle streamChat(ChatRequest request, StreamCallback callback) {
    List<ModelTarget> targets = selector.selectChatCandidates(Boolean.TRUE.equals(request.getThink()
        if (CollUtil.isEmpty(targets)) {
            throw new RemoteException(STREAM_NO_PROVIDER_MESSAGE);
        }

    String label = ModelCapability.CHAT.getDisplayName();
    Throwable lastError = null;

    for (ModelTarget target : targets) {
        ChatClient client = resolveClient(target, label);
        if (client == null) {
            continue;
        }
        if (!healthStore.allowCall(target.id())) {
            continue;
        }

        ProbeStreamBridge bridge = new ProbeStreamBridge(callback);

        StreamCancellationHandle handle;
        try {
            handle = client.streamChat(request, bridge, target);
        } catch (Exception e) {
            healthStore.markFailure(target.id());
            lastError = e;
            log.warn("{} 流式请求启动失败，切换下一个模型。modelId: {}, provider: {}",
                label, target.id(), target.candidate().getProvider(), e);
            continue;
        }
        if (handle == null) {
            healthStore.markFailure(target.id());
            lastError = new RemoteException(STREAM_START_FAILED_MESSAGE, BaseErrorCode.REMOTE_ERROR);
            log.warn("{} 流式请求未返回取消句柄，切换下一个模型。modelId: {}, provider: {}",
                label, target.id(), target.candidate().getProvider());
            continue;
        }

        ProbeStreamBridge.ProbeResult result = awaitFirstPacket(bridge, handle, callback);
```

```
if (result.isSuccess()) {
    healthStore.markSuccess(target.id());
    return handle;
}

// 失败处理
healthStore.markFailure(target.id());
handle.cancel();

lastError = buildLastErrorAndLog(result, target, label);
}

// 所有模型都失败了，通知客户端错误
throw notifyAllFailed(callback, lastError);
}
```

代码不短，逐段拆解。

3. 执行流程逐段讲解

选择候选列表： `selector.selectChatCandidates()` 获取按优先级排序的候选列表（第二篇讲过）。空列表直接抛异常——没有可用模型，无法继续。

遍历候选： 和 `executeWithFallback` 一样的 `for` 循环结构，逐个尝试候选。

客户端查找 + 熔断检查： `resolveClient(target, label)` 从 `clientsByProvider` Map 中按供应商查找客户端，`null` 则 `warn + continue`。`healthStore.allowCall(target.id())` 做熔断检查（第三篇讲过），`false` 则 `continue`。这两步和同步路由完全一样。

这里可能有疑问：`healthStore.allowCall()` 已经做了熔断检查，为什么还需要后面的首包探测？因为两者管的维度不同。`ModelHealthStore` 基于历史统计——过去 `N` 次调用的成功失败比例，它回答的是这个模型最近整体靠不靠谱。而 `ProbeStreamBridge` 探测的是当前这一次请求的实时状态——一个模型可能过去 `100` 次调用全部成功（熔断器 `CLOSED`，放行），但这一次恰好赶上 `API` 限流、服务滚动重启、本地模型崩溃，请求实际会失败。熔断器是粗筛，过滤掉已知不健康的模型，避免浪费时间去探测一个大概率失败的候选。首包探测是细筛，对熔断器放行的模型做当次请求的实时验证。两者互补，不是替代。

创建探测桥接器——这是流式路由独有的：

```
ProbeStreamBridge bridge = new ProbeStreamBridge(callback);
```

每个候选都创建独立的 `ProbeStreamBridge`。为什么不能复用？因为内部的 `CompletableFuture` 是一次性的——`complete()` 之后状态不可逆。缓冲列表和 `committed` 状态也是一次性的。每次重试是全新的探测过程，必须用全新的桥接器。

启动流式调用：

```
handle = client.streamChat(request, bridge, target);
```

注意传入的是 `bridge` 而非真实 `callback`。这是整个 `probe-and-commit` 模式的关键——供应商的所有回调事件都先经过 `bridge`，不直接到达真实 `callback`。

启动异常处理： `client.streamChat()` 本身可能抛同步异常（比如前置校验失败、`StreamAsyncExecutor` 线程池拒绝等）。这类异常不需要等首包，直接 `markFailure + continue` 到下一个候选。

handle null 检查： 防御性编程，`streamChat()` 理论上不应该返回 `null`，但加一层检查更安全。

阻塞等待首包：

```
ProbeStreamBridge.ProbeResult result = awaitFirstPacket(bridge, handle, callback);
```

这一行是路由线程的阻塞点——在这里等待异步线程的首包信号，最多等 `60` 秒。详细机制后面讲 `ProbeStreamBridge` 时展开。

结果判断——四种情况对应不同处理：

```
if (result.isSuccess()) {
    healthStore.markSuccess(target.id()); // 标记模型健康
    return handle;                       // 返回取消句柄给调用方
}
```

```
// 失败
healthStore.markFailure(target.id()); // 标记模型失败
handle.cancel(); // 取消当前流式连接
lastError = buildLastErrorAndLog(result, target, label); // 记日志, continue 到下一个
```

成功路径: markSuccess → return handle。awaitFirstPacket() 内部已经自动完成了 commit——缓冲的事件已刷给真实 callback, 后续 token 直通不再缓冲。返回的 handle 是当前供应商的取消句柄, 调用方可以用它停止生成。

失败路径: markFailure → handle.cancel() → continue。取消当前供应商的连接 (释放资源), 切换到下一个候选。bridge 里缓冲的事件 (比如错误的 onError) 不会被 commit, 直接随 bridge 对象一起被 GC 回收——真实 callback 永远看不到它们。

所有候选失败:

```
throw notifyAllFailed(callback, lastError);
```

循环结束都没有成功返回, 说明所有候选都失败了。notifyAllFailed 做两件事: callback.onError() 通知前端显示错误信息, 然后抛 RemoteException 通知调用链上层。

4. awaitFirstPacket 方法

```
private ProbeStreamBridge.ProbeResult awaitFirstPacket(ProbeStreamBridge bridge,
                                                         StreamCancellationHandle handle,
                                                         StreamCallback callback) {
    try {
        return bridge.awaitFirstPacket(FIRST_PACKET_TIMEOUT_SECONDS, TimeUnit.SECONDS);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        handle.cancel();
        RemoteException interruptedException =
            new RemoteException(STREAM_INTERRUPTED_MESSAGE, e, BaseErrorCode.REMOTE_ERROR);
        callback.onError(interruptedException);
        throw interruptedException;
    }
}
```

正常情况下就是一行 bridge.awaitFirstPacket(60, SECONDS)——阻塞等待, 最多 60 秒。

InterruptedException 的处理比较特殊: 恢复中断标志 → 取消流 → 通知错误 → 抛异常。注意这里不是 continue 到下一个候选, 而是直接 throw 中止整个 streamChat 方法。因为线程中断通常意味着应用正在关闭或者上层明确要求中止 (比如 Tomcat shutdown), 这种情况下不应该继续重试。

5. 失败处理和错误通知

```
private Throwable buildLastErrorAndLog(ProbeStreamBridge.ProbeResult result,
                                       ModelTarget target, String label) {
    switch (result.getType()) {
        case ERROR -> {
            Throwable error = result.getError() != null
                ? result.getError()
                : new RemoteException("流式请求失败", BaseErrorCode.REMOTE_ERROR);
            log.warn("{} 失败模型: modelId={}, provider={}, 原因: 流式请求失败, 切换下一个模型",
                    label, target.id(), target.candidate().getProvider(), error);
            return error;
        }
        case TIMEOUT -> {
            RemoteException timeout =
                new RemoteException(STREAM_TIMEOUT_MESSAGE, BaseErrorCode.REMOTE_ERROR);
            log.warn("{} 失败模型: modelId={}, provider={}, 原因: 流式请求超时, 切换下一个模型",
                    label, target.id(), target.candidate().getProvider());
            return timeout;
        }
        case NO_CONTENT -> {

```

```

        RemoteException noContent =
            new RemoteException(STREAM_NO_CONTENT_MESSAGE, BaseErrorCode.REMOTE_ERROR);
        log.warn("{} 失败模型: modelId={}, provider={}, 原因: 流式请求无内容完成, 切换下一个模型",
            label, target.id(), target.candidate().getProvider());
        return noContent;
    }
    default -> {
        RemoteException unknown =
            new RemoteException("流式请求失败", BaseErrorCode.REMOTE_ERROR);
        log.warn("{} 失败模型: modelId={}, provider={}, 原因: 流式请求失败 (未知类型), 切换下一个模型",
            label, target.id(), target.candidate().getProvider());
        return unknown;
    }
}
}

```

每种失败类型记录不同的日志，构建对应的异常。返回的异常赋给 `lastError`，如果所有候选都失败，最终会被 `notifyAllFailed` 包装后抛出。

```

private RemoteException notifyAllFailed(StreamCallback callback, Throwable lastError) {
    RemoteException finalException = new RemoteException(
        STREAM_ALL_FAILED_MESSAGE,
        lastError,
        BaseErrorCode.REMOTE_ERROR
    );
    callback.onError(finalException);
    return finalException;
}

```

`notifyAllFailed` 做了双重通知：`callback.onError()` 让前端知道失败了（显示大模型调用失败，请稍后再试...），`throw RemoteException` 让调用链上层知道失败了（比如 `Controller` 层的全局异常处理器）。两个通道都不遗漏，确保错误信息传达到位。

ProbeStreamBridge：探测桥接器

1. 完整代码

```

final class ProbeStreamBridge implements StreamCallback {

    private final StreamCallback downstream;
    private final CompletableFuture<ProbeResult> probe = new CompletableFuture<>();
    private final Object lock = new Object();
    private final List<Runnable> buffer = new ArrayList<>();
    private volatile boolean committed;

    ProbeStreamBridge(StreamCallback downstream) {
        this.downstream = downstream;
    }

    @Override
    public void onContent(String content) {
        probe.complete(ProbeResult.success());
        bufferOrDispatch(() -> downstream.onContent(content));
    }

    @Override
    public void onThinking(String content) {
        probe.complete(ProbeResult.success());
        bufferOrDispatch(() -> downstream.onThinking(content));
    }
}

```

```
@Override
public void onComplete() {
    probe.complete(ProbeResult.noContent());
    bufferOrDispatch(downstream::onComplete);
}

@Override
public void onError(Throwable t) {
    probe.complete(ProbeResult.error(t));
    bufferOrDispatch(() -> downstream.onError(t));
}

ProbeResult awaitFirstPacket(long timeout, TimeUnit unit) throws InterruptedException {
    ProbeResult result;
    try {
        result = probe.get(timeout, unit);
    } catch (TimeoutException e) {
        return ProbeResult.timeout();
    } catch (ExecutionException e) {
        return ProbeResult.error(e.getCause());
    }

    if (result.isSuccess()) {
        commit();
    }

    return result;
}

private void commit() {
    synchronized (lock) {
        if (committed) {
            return;
        }
        committed = true;
        buffer.forEach(Runnable::run);
    }
}

private void bufferOrDispatch(Runnable action) {
    boolean dispatchNow;
    synchronized (lock) {
        dispatchNow = committed;
        if (!dispatchNow) {
            buffer.add(action);
        }
    }
    if (dispatchNow) {
        action.run();
    }
}

@Getter
static class ProbeResult {

    enum Type { SUCCESS, ERROR, TIMEOUT, NO_CONTENT }

    private final Type type;
    private final Throwable error;

    private ProbeResult(Type type, Throwable error) {
        this.type = type;
        this.error = error;
    }
}
```

```

        static ProbeResult success() { return new ProbeResult(Type.SUCCESS, null); }
        static ProbeResult error(Throwable t) { return new ProbeResult(Type.ERROR, t); }
        static ProbeResult timeout() { return new ProbeResult(Type.TIMEOUT, null); }
        static ProbeResult noContent() { return new ProbeResult(Type.NO_CONTENT, null); }

        boolean isSuccess() { return type == Type.SUCCESS; }
    }
}

```

旧版实现把探测和缓冲拆成了两个类：FirstPacketAwaiter（基于 CountDownLatch + 4 个原子变量）负责跨线程信号同步，ProbeBufferingCallback（基于 BufferedEvent record + EventType enum + dispatch switch）负责事件缓冲。调用方需要手动创建两个对象并协调它们——先 await，成功后再手动 commit。

重构后合并为 ProbeStreamBridge 一个类，两个核心简化：

信号层：CompletableFuture<ProbeResult> 替代 CountDownLatch + AtomicBoolean hasContent + AtomicBoolean eventFired + AtomicReference<Throwable> error。CompletableFuture.complete() 天然有 CAS 语义（只有第一次调用生效），自带阻塞等待和超时，一个字段完成四个字段的工作

缓冲层：List<Runnable> + lambda 替代 BufferedEvent record + EventType enum + dispatch() switch。每个回调方法直接用 lambda 捕获下游操作（如 () -> downstream.onContent(content)），省去了"拆成数据再还原回调"的中间表示

下面逐个模块拆解。

2. CompletableFuture 信号机制

ProbeStreamBridge 的探测信号只用一个字段：

```
private final CompletableFuture<ProbeResult> probe = new CompletableFuture<>();
```

四个回调方法各自调 probe.complete()，传入对应的探测结果：

```

public void onContent(String content) {
    probe.complete(ProbeResult.success());          // 收到内容 → 探测成功
    bufferOrDispatch(() -> downstream.onContent(content));
}

public void onThinking(String content) {
    probe.complete(ProbeResult.success());          // 收到思考 → 探测成功
    bufferOrDispatch(() -> downstream.onThinking(content));
}

public void onComplete() {
    probe.complete(ProbeResult.noContent());        // 没内容就结束了 → 无内容
    bufferOrDispatch(downstream::onComplete);
}

public void onError(Throwable t) {
    probe.complete(ProbeResult.error(t));           // 出错 → 探测失败
    bufferOrDispatch(() -> downstream.onError(t));
}

```

CompletableFuture.complete() 有一个关键特性：**只有第一次调用生效**，后续调用被忽略。这正好满足首包探测的语义——只关心第一个到达的信号。如果 onContent 先到，probe 被设为 SUCCESS，后续的 onComplete 或 onError 再调 complete() 就无效了。如果 onError 先到，probe 被设为 ERROR，后续的 onContent 也无效。

路由线程通过 probe.get(timeout, unit) 阻塞等待结果：

```

ProbeResult awaitFirstPacket(long timeout, TimeUnit unit) throws InterruptedException {
    ProbeResult result;
    try {
        result = probe.get(timeout, unit);
    } catch (TimeoutException e) {

```



```
        return ProbeResult.timeout();
    } catch (ExecutionException e) {
        return ProbeResult.error(e.getCause());
    }

    if (result.isSuccess()) {
        commit();
    }
    return result;
}
```

probe.get() 阻塞直到 complete() 被调用或超时。超时抛 TimeoutException，返回 TIMEOUT。成功时**内部自动** commit()——调用方不需要手动提交，不可能忘调或错调。

2.1 为什么 onContent 和 onThinking 都发 success 信号

对首包探测来说，收到思考内容和正式内容的语义是一样的——都表示供应商已经开始响应了，连接是通的，模型正在工作。不管先来的是 reasoning_content 还是 content，只要有数据到达，探测就算成功。

2.2 NO_CONTENT 是什么场景

onComplete() 发出 ProbeResult.noContent() 信号。这是一种边缘但真实存在的场景——某些大模型在内容被安全过滤拦截后，可能直接发 finish_reason: "stop" 而没有任何 content。这时 onComplete 被调用，但没有实际内容推送过。路由层把这种情况视为失败——一个不产出任何内容的回答对用户没有意义，不如切换到下一个供应商试试。

2.3 四种结果的优先级

CompletableFuture 的“第一个 complete 生效”语义天然决定了优先级——谁先到，谁就是最终结果：

结果类型	触发条件	含义	路由层处理
SUCCESS	onContent 或 onThinking 先到	收到了内容，探测成功	awaitFirstPacket 内部自动 commit + markSuccess + 返回 handle
ERROR	onError 先到	供应商报错	markFailure + cancel + 切换下一个
TIMEOUT	probe.get() 超时	60 秒内没有收到任何信号	markFailure + cancel + 切换下一个
NO_CONTENT	onComplete 先到且之前无 content	收到了结束信号但没有实际内容	markFailure + cancel + 切换下一个

3. 装饰器模式与两阶段设计

ProbeStreamBridge 实现了 StreamCallback 接口，包装了真实的 downstream 回调。从 doStream 的视角看，它和普通的 StreamCallback 没有区别——照样调 onContent、onThinking、onComplete、onError。doStream 完全不知道自己的回调被拦截了。

这就是装饰器模式的作用：在不修改 doStream 代码的情况下，插入了缓冲和探测逻辑。

两阶段的切换点是 committed 标志：

缓冲阶段 (committed = false) ——所有回调事件被封装为 lambda，存入 buffer 列表。不调用 downstream 的任何方法。同时每个回调方法都通过 probe.complete() 发出信号，让路由线程知道发生了什么。

直通阶段 (committed = true) ——awaitFirstPacket() 探测成功后内部调 commit()，committed 变为 true。后续到达的事件不再缓冲，lambda 直接执行。commit() 本身也会把之前缓冲的 lambda 逐个执行，刷给 downstream。

4. lambda 缓冲：用 Runnable 替代类型枚举

旧版用 BufferedEvent record + EventType enum 统一建模四种事件，dispatch 时用 switch 还原为对应的回调调用。这本质上是把回调拆成数据，再从数据还原回调——多了一层中间表示。

旧版本重构代码提交：[refactor\(chat\): 优化流式首包探测桥接器实现](#)

新版直接用 lambda 捕获回调动作：

```
bufferOrDispatch(() -> downstream.onContent(content));
bufferOrDispatch(() -> downstream.onThinking(content));
bufferOrDispatch(downstream::onComplete);
bufferOrDispatch(() -> downstream.onError(t));
```

每个 lambda 就是一个待执行的 Runnable，缓冲在 List<Runnable> 中。commit 时遍历列表执行每个 lambda：

```
private void commit() {
    synchronized (lock) {
        if (committed) {
            return;
        }
        committed = true;
        buffer.forEach(Runnable::run);
    }
}
```

不需要 BufferedEvent record、EventType enum、dispatch switch——lambda 本身就包含了所有信息。

5. bufferOrDispatch 和 commit 的线程安全

这是整个桥接器最精巧的部分。bufferOrDispatch 在异步线程（modelStreamExecutor）上被调用，commit 在 awaitFirstPacket 中被触发（路由线程），两者可能并发执行。

```
private void bufferOrDispatch(Runnable action) {
    boolean dispatchNow;
    synchronized (lock) {
        dispatchNow = committed;
        if (!dispatchNow) {
            buffer.add(action);
        }
    }
    if (dispatchNow) {
        action.run();
    }
}
```

两个方法用同一把 lock，synchronized 保证它们不会交叉执行。为什么需要这个保证？因为不加锁会出三个问题——**事件丢失、并发修改异常、事件乱序**。下面用模型回答 AirPods Pro 2 的场景逐一说明。

5.1 问题一：事件丢失

假设模型已经推了第一个 token "AirPods"（触发了首包探测成功），路由线程正准备 commit()，同时异步线程收到了第二个 token "Pro" 正在调 bufferOrDispatch()。两个线程并发操作共享的 committed 标志和 buffer 列表。

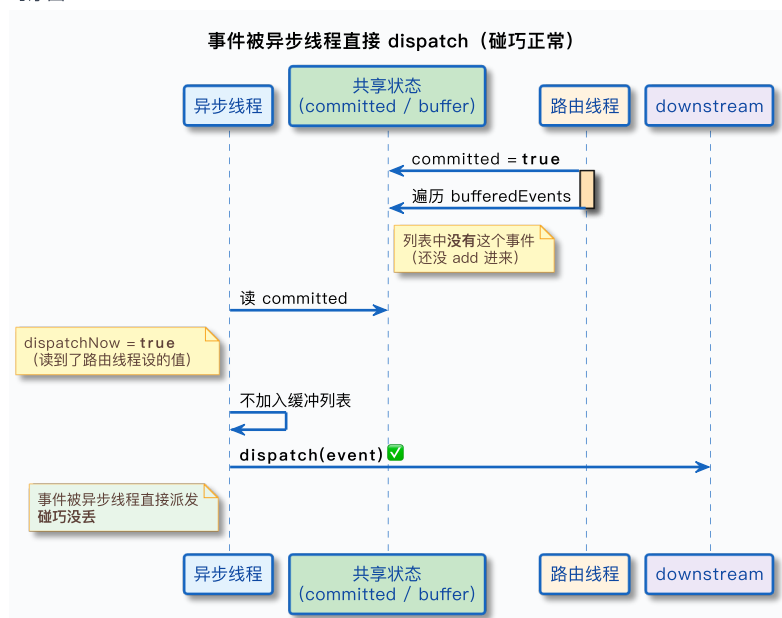
如果 bufferOrDispatch 不加 synchronized：

```
// ⚠️ 没加锁的 bufferOrDispatch
private void bufferOrDispatch(Runnable action) {
    boolean dispatchNow = committed;    // ← 第一步：读 committed
    if (!dispatchNow) {
        buffer.add(action);            // ← 第二步：加入列表
    }
    if (dispatchNow) {
        action.run();
    }
}
```

第一步和第二步之间不是原子的——commit() 可以插进来执行：

```
异步线程 (收到 " Pro")  
_____  
  
// 第一步: 读 committed  
dispatchNow = committed  
// 读到 false (commit 还没执行)  
  
// 第二步: 加入列表  
buffer.add(" Pro" 的 lambda)  
// 加进去了, 但 commit 已经遍历完了  
// 下面 dispatchNow = false, 不走 run  
// commit 也不会再来遍历了  
// → " Pro" 丢了!  
  
路由线程 (commit)  
_____  
  
synchronized(lock) {  
    committed = true  
    遍历 buffer → 只有 "AirPods" 的 lambda  
    run("AirPods") → 前端收到 "AirPods"  
    // " Pro" 还没加进来, 遍历不到  
}
```

时序图:



前端看到的回答变成了"AirPods 2", 中间的 " Pro" 消失了。

加了 synchronized 之后, 两个方法互斥, 只有两种可能的执行顺序:

顺序 A——异步线程先拿到锁 (" Pro" 先加入列表):

```
异步线程先执行:  
synchronized(lock) {  
    dispatchNow = committed    // false  
    buffer.add(" Pro" 的 lambda) // " Pro" 在列表里了  
}
```

```
路由线程后执行:  
synchronized(lock) {  
    committed = true  
    遍历 buffer → "AirPods", " Pro"  
    run("AirPods") → 前端收到  ✓  
    run(" Pro")    → 前端收到  ✓  
}
```

顺序 B——路由线程先拿到锁 (commit 先完成):

路由线程先执行：

```
synchronized(lock) {  
    committed = true  
    遍历 buffer → 只有 "AirPods"  
    run("AirPods") → 前端收到 ✓  
}
```

异步线程后执行：

```
synchronized(lock) {  
    dispatchNow = committed    // true, 已经 commit 了  
}  
action.run()                  → 前端收到 " Pro" ✓    // 走直通路
```

不管谁先谁后，每个事件都恰好被执行一次，不丢失。

5.2 问题二：ArrayList 并发修改异常

buffer 是 ArrayList，不是线程安全的。commit() 用 forEach 遍历列表（内部创建迭代器），如果同时 bufferOrDispatch() 往里 add()：

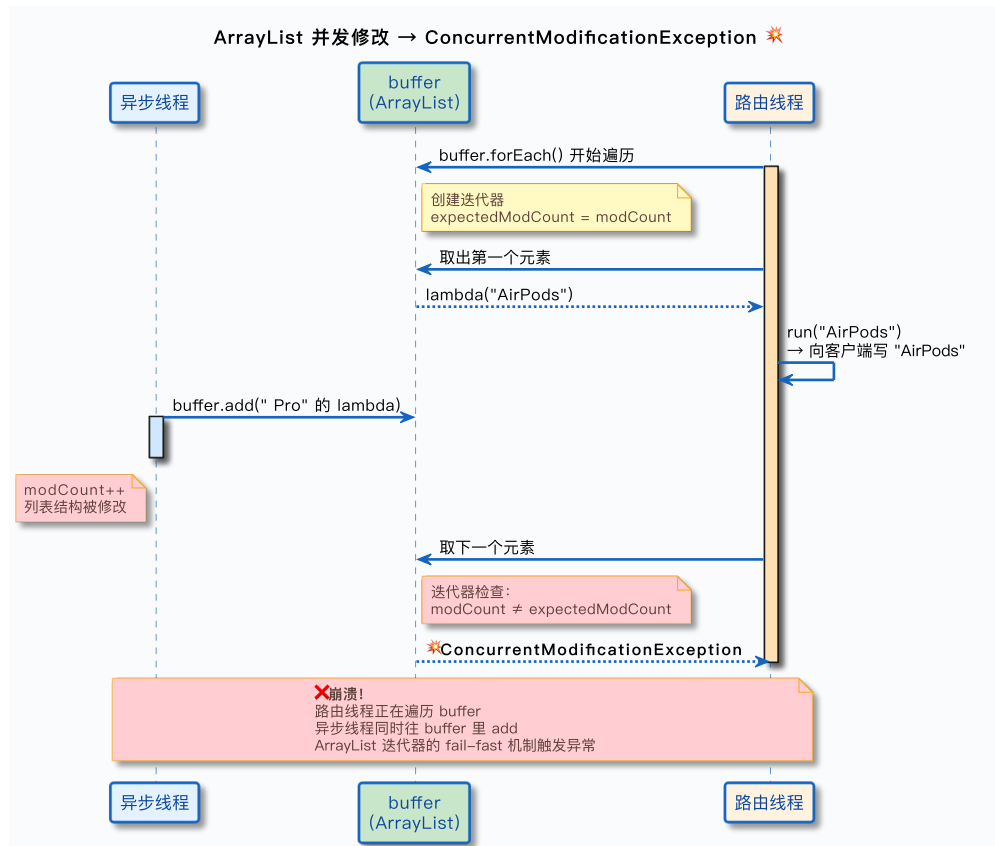
异步线程

```
buffer.add(" Pro" 的 lambda)
```

路由线程

```
buffer.forEach(Runnable::run) {  
    // 迭代器正在遍历  
    run("AirPods")  
    // ← ArrayList 结构被修改  
    // 迭代器检测到 modCount 变了  
    // → ConcurrentModificationException ✖  
}
```

时序图如下：



ArrayList 的迭代器有 fail-fast 机制——遍历过程中如果列表的结构被其他操作修改（添加、删除），迭代器立即抛 ConcurrentModificationException。即使侥幸没抛异常，ArrayList 底层数组在扩容时可能被替换，并发读写会导致数据错乱（读到未初始化的元素、数组越界等）。

synchronized 让遍历和添加互斥——commit() 持锁遍历时，异步线程拿不到锁进不来，不会并发修改。

5.3 问题三：事件乱序

注意 `commit` 的遍历执行是在 `synchronized(lock)` 块内部执行的，而不是像常见的并发模式那样先拍照再在锁外遍历。这是有意为之——防止事件乱序。

假设用锁外执行的写法：

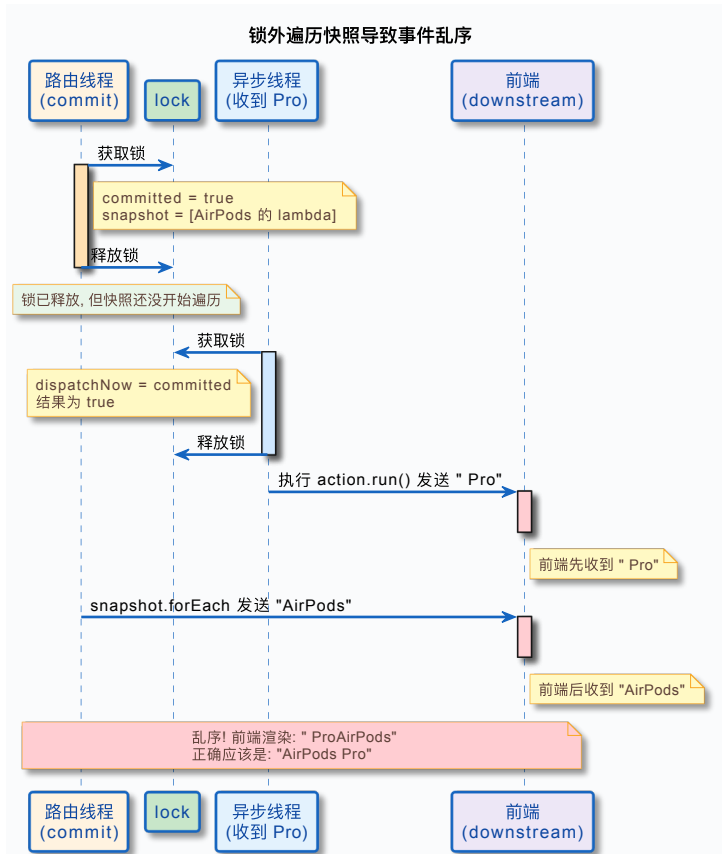
```
// ⚠️ 有乱序风险的写法
void commit() {
    List<Runnable> snapshot;
    synchronized (lock) {
        committed = true;
        snapshot = new ArrayList<>(buffer);
    }

    // 锁释放了，异步线程可以进来了
    snapshot.forEach(Runnable::run);
}
```

假设 `buffer` 里缓冲了 "AirPods" 的 `lambda`，异步线程刚好收到了 "Pro"：

路由线程 (commit)	异步线程 (收到 "Pro")
<pre>synchronized(lock) { committed = true snapshot = ["AirPods" 的 lambda] }</pre> // 锁释放了!	<pre>synchronized(lock) { dispatchNow = committed // true }</pre> <code>action.run()</code> → 前端先收到 "Pro"
<code>snapshot.forEach(Runnable::run)</code>	→ 前端后收到 "AirPods"

时序如下：



"AirPods" 本来应该在 "Pro" 前面（它是更早到达的缓冲事件），但因为 `commit` 释放锁之后、遍历快照之前，异步线程的 "Pro" 抢先执行了。前端渲染的文本变成了 "ProAirPods 2"——乱了。

把执行放在锁内就杜绝了这个问题——commit 持锁期间遍历并执行，异步线程被锁挡住无法抢先。等 commit 释放锁时，"AirPods" 已经执行完了，异步线程拿到锁后看到 committed == true，走直通路径执行 " Pro"，顺序正确。

代价是 commit 持锁时间变长了——锁内要执行 lambda，而 lambda 调的是 downstream 的方法，可能涉及 I/O（比如通过 SseEmitter 推送给前端）。但这个代价可以接受：commit 只执行一次，缓冲列表里通常只有 1~2 个事件（首包探测成功时只缓冲了第一个 token），持锁时间极短。而乱序一旦发生，用户看到的就是文本跳乱，这比短暂的锁竞争严重得多。正确性优先于性能。

5.4 小结

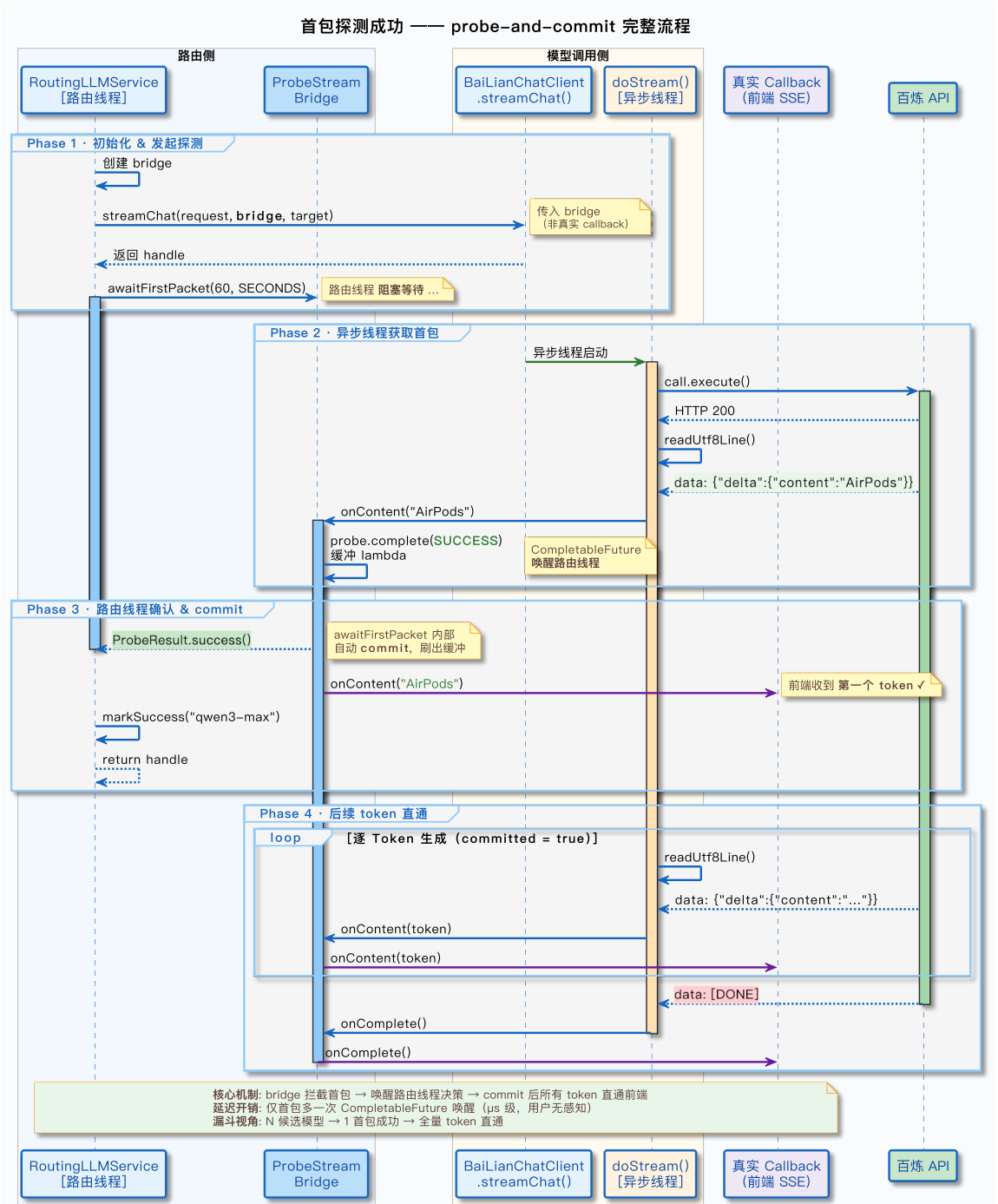
三个问题，一把锁全解决：

问题	根因	锁怎么解决的
事件丢失	"读 committed + 加列表"不是原子操作，commit 插进来后两边都不执行	锁保证 check-then-act 原子性，事件要么在列表里被 commit 遍历到，要么看到 committed = true 走直通
并发修改异常	ArrayList 非线程安全，遍历和添加并发执行	锁让遍历和添加互斥
事件乱序	commit 锁外执行时，新事件抢先执行	commit 锁内执行，新事件被锁挡住，等 commit 完成后才走直通

完整场景走查

1. 场景一：首包探测成功（百炼正常响应）

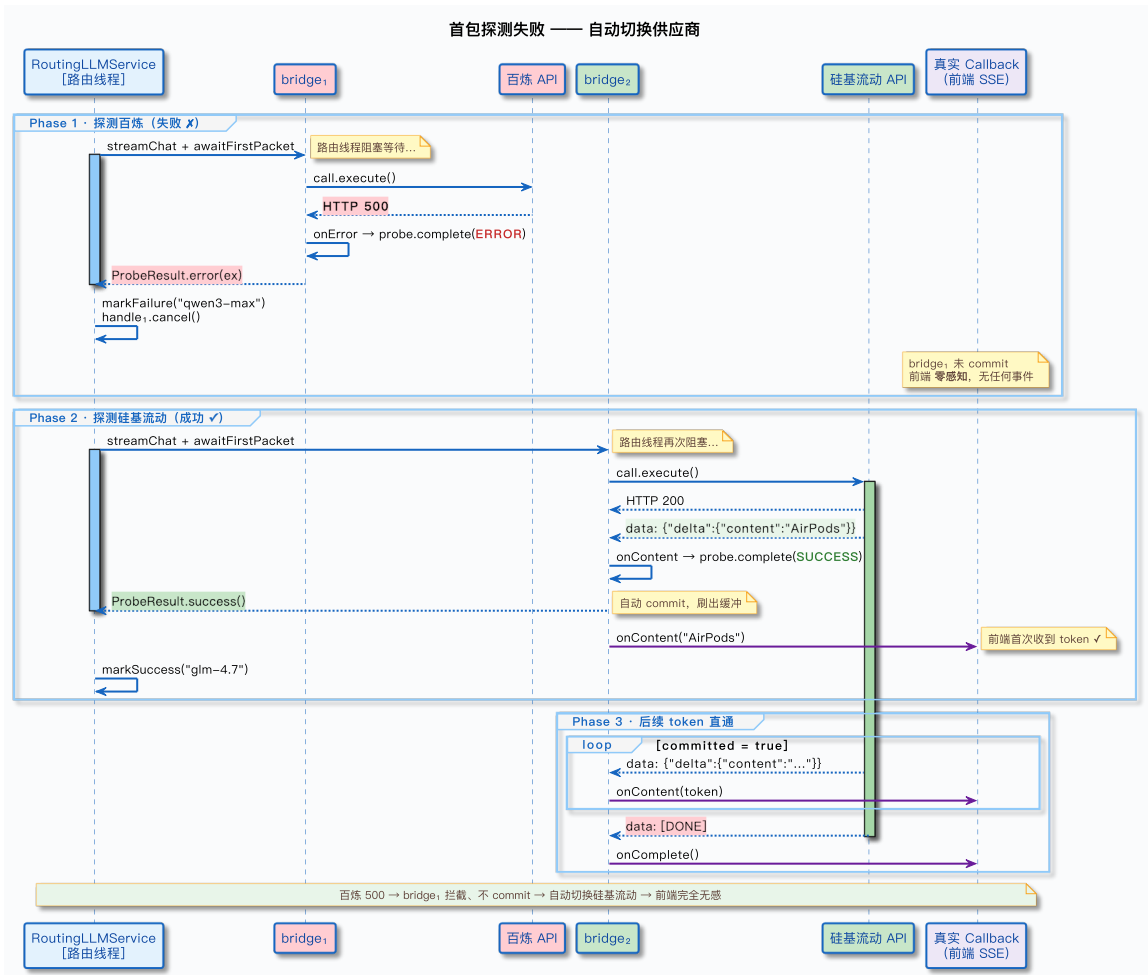
候选列表：[qwen3-max, glm-4.7, qwen-plus, qwen3-local]。用户问"AirPods Pro 2 的保修期是多久？"。



从用户视角看：前端在 `awaitFirstPacket()` 内部自动 `commit` 时收到第一个 token "AirPods"，后续 token 持续流入，体验和没有探测机制一样流畅。整个探测过程（创建 `bridge`、等待首包、自动 `commit`）只增加了极微小的延迟——从异步线程收到第一个 token 到路由线程被唤醒再到 `commit` 刷出，通常在毫秒级别。

2. 场景二：百炼失败，切换到硅基流动

百炼 API 返回 HTTP 500，路由层自动切换到下一个候选。



关键点：在整个切换过程中，真实 callback 没有收到任何来自百炼的事件。bridge₁ 缓冲了百炼的 onError，但因为探测失败 awaitFirstPacket() 没有调 commit，这个错误不会转发给前端。bridge₁ 连同它缓冲的事件一起被丢弃。前端只看到硅基流动的正常回答，就像百炼从来没有被尝试过一样。

用户唯一能感知到的差异是首字延迟略长——因为多了一次百炼的探测失败时间（HTTP 连接 + 等待 500 响应）。但比起看到重复或断裂的内容，多等一两秒完全可以接受。

3. 场景三：所有供应商都失败

假设百炼返回 HTTP 500，硅基流动连接超时（60 秒），Ollama 本地服务没启动（连接拒绝），候选列表遍历完毕：

- qwen3-max（百炼）：doStream 收到 HTTP 500 → bridge.onError() → ProbeResult.error() → markFailure + cancel + continue
- glm-4.7（硅基流动）：doStream 阻塞在 call.execute() → 60 秒超时 → bridge.awaitFirstPacket() 抛 TimeoutException → ProbeResult.timeout() → markFailure + cancel + continue
- qwen-plus（百炼）：同 1，百炼还是 500
- qwen3-local（Ollama）：client.streamChat() 抛 ConnectException（Ollama 没启动）→ catch 块 markFailure + continue

循环结束，执行：

```
throw notifyAllFailed(callback, lastError);
```

callback.onError(RemoteException("大模型调用失败，请稍后再试...")) → 前端显示错误信息。抛出 RemoteException 通知调用链上层。

4. 场景四：本地模型承压宕机——首包探测的核心场景

前三个场景（HTTP 500、连接超时、连接拒绝）虽然在机制上被首包探测捕获，但它们本质上是快速失败——HTTP 层面就能判断出问题，不需要等首包来发现。首包探测真正要解决的问题比这些更隐蔽：**HTTP 连接成功了（200 OK），模型也接受了请求，但在生成第一个 token 之前崩溃了。**

这种故障在云 API 上几乎不会出现——云端供应商有负载均衡保护，单节点崩溃由平台自行处理，请求方看到的要么是快速的 HTTP 错误，要么是正常响应。但本地部署的模型（Ollama、vLLM）直接暴露在应用面前，没有中间层保护，承压宕机、显存溢出、模型加载失败都是常事——而且故障时机往往不是在连接建立时，而是在模型开始处理请求之后。

假设候选列表是 [qwen3-local（Ollama 本地），qwen3-max（百炼云）]。用户发了一段很长的文本让模型总结，Ollama 收到请求后开始加载上下文，GPU 显存不够，进程 OOM 崩溃：

1. qwen3-local (Ollama): client.streamChat() 正常返回 handle (HTTP 连接建立成功, Ollama 接受了请求) → 路由线程阻塞在 bridge.awaitFirstPacket(60, SECONDS) → 异步线程 call.execute() 拿到 HTTP 200 (Ollama 开始处理) → readUtf8Line() 阻塞等待第一个 token → Ollama 进程 OOM 崩溃, 连接断开 → readUtf8Line() 返回 null → doStream 抛 ModelClientException("流式响应异常结束") → bridge.onError(ex) → probe.complete(ERROR) → 路由线程被唤醒 → ProbeResult.error() → markFailure + cancel + continue
2. qwen3-max (百炼): 创建新的 bridge2 → streamChat() → 百炼正常响应 → 首包探测成功 → commit() → 前端开始收到 token

前端完全不知道 Ollama 崩溃了。它只是等了稍微久一点 (Ollama 连接建立 + 等待崩溃 + 百炼首包延迟), 然后收到百炼的正常回答。注意这个场景和 ModelHealthStore 的区别——Ollama 之前一直正常工作 (熔断器 CLOSED, 放行), 这次是因为输入文本太长导致 OOM, 属于偶发故障。熔断器基于历史统计, 只能在连续多次失败后才会打开, 无法预防第一次崩溃。首包探测恰好弥补了这个空白: 不依赖历史数据, 直接验证当前请求能不能拿到第一个 token。

5. 首包之后的故障怎么办

首包探测覆盖的是首包到达之前的故障。一个自然的问题是: 如果 commit() 之后供应商中途挂了怎么办?

答案是: **不切换, 直接把错误传给前端。**

这不是能力缺失, 而是设计决策。原因回到之前讲的——awaitFirstPacket() 内部自动 commit 之后, 真实 callback 已经收到了 token, 前端已经渲染了部分内容。如果这时候切换到另一个供应商重新生成, 新供应商的回答会从头开始, 前端看到的就是一段残片 + 一段完整回答拼在一起, 内容重复且断裂。

当然也不是不行, 比如像豆包的一些机制, 虽然已经展示给用户了, 但是可以清空掉再用新模型的回复展示。这种用户就会很奇怪, 为什么数据还会变化。

所以 commit() 是一个不可逆的分界线:

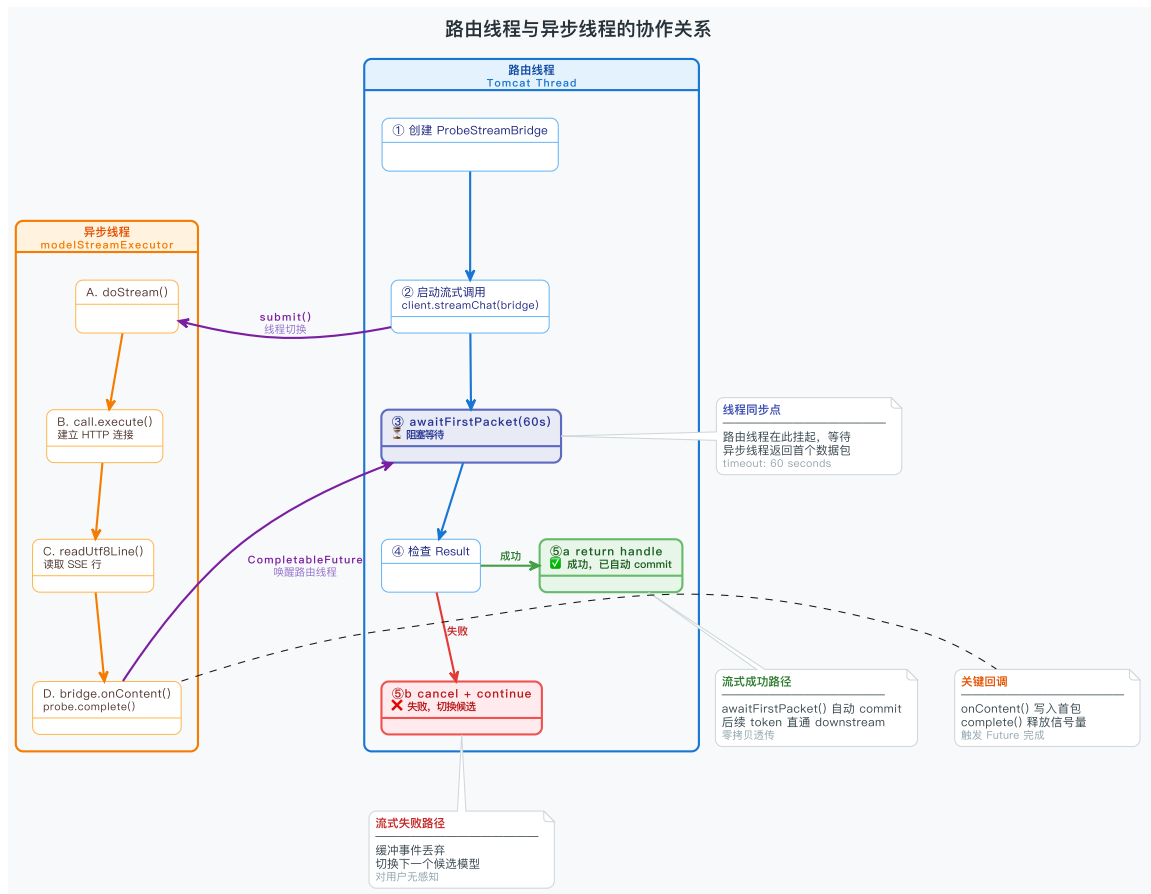
commit 之前——缓冲阶段, 真实 callback 没收到任何数据, 可以随意切换, 前端无感知

commit 之后——直通阶段, 数据已经推给前端。出错就 onError, 让前端提示用户重试。这比拼接两个供应商的残片好得多

这也意味着首包探测的防护范围是有边界的: 它覆盖的是 HTTP 连接成功但首包到达前模型崩溃这个窗口, 不能防住中途网络断开或模型生成到一半崩溃。但这恰好是本地部署模型最容易出故障的阶段——模型收到请求后开始加载上下文、分配显存, 这个过程最容易 OOM。一旦模型开始稳定输出 token, 中途崩溃的概率远低于首包失败的概率。

两个线程的完整协作模型

把路由线程和异步线程的协作关系画一张全景图, 帮助理解各组件之间的关系:



路由线程负责决策（选谁、成功还是失败、提交还是切换），异步线程负责执行（建连接、读数据、调回调）。两者通过 `ProbeStreamBridge` 协调——`CompletableFuture` 传递首包信号，`synchronized + committed` 管理缓冲到直通的数据流切换。

小结

回顾这一篇的核心要点：

首包探测的核心场景：本地部署模型（Ollama、vLLM）在承压时宕机——HTTP 连接建立成功、模型接受了请求，但在生成第一个 token 之前崩溃（GPU 显存溢出、进程 OOM）。这个窗口期是 HTTP 错误处理和熔断器都覆盖不了的：HTTP 层没有报错，熔断器历史数据显示健康，错误发生在异步线程里。云 API 有负载均衡保护，出现这种情况的概率极低

流式路由的核心挑战：`executeWithFallback` 的同步 `try-catch` 模式无法用于流式调用——异步回调模式下，故障发生在方法返回之后，`try-catch` 捕获不到。而且如果直接传真实 `callback`，供应商推了数据后再报错，数据无法撤回，切换供应商会导致前端内容重复

`probe-and-commit` 模式：先用 `ProbeStreamBridge` 拦截和缓冲回调事件，内部通过 `CompletableFuture` 阻塞路由线程等首包信号。`awaitFirstPacket()` 成功后内部自动 `commit`，刷出缓冲并切换为直通模式；失败则 `cancel()` 取消连接、丢弃缓冲、切换下一个候选。真实 `callback` 在 `commit` 之前不会收到任何数据，故障转移对前端完全透明。`commit` 之后如果中途出错，不切换，直接 `onError` 让前端提示重试——因为数据已推送，切换会导致内容重复断裂

`ProbeStreamBridge` 同时承担回调缓冲和首包等待的职责，`synchronized(lock)` 保护缓冲阶段和直通阶段的切换，保证事件不丢失、不重复、不乱序。`commit()` 在锁内执行 `List<Runnable> + lambda` 的 `dispatch`——虽然持锁时间略长，但杜绝了锁外 `dispatch` 导致的事件乱序问题（缓冲事件和新到达事件交叉 `dispatch`），正确性优先于性能

`ProbeStreamBridge` 用 `CompletableFuture` 实现路由线程和异步回调线程的同步。`CompletableFuture.complete()` 天然 CAS，保证只触发一次唤醒。四种结果（`SUCCESS / ERROR / TIMEOUT / NO_CONTENT`）覆盖所有可能的首包探测场景，`ERROR` 优先级最高确保有错误就视为失败

`RoutingLLMService.streamChat()` 自己实现遍历 + 故障转移（没有用 `executeWithFallback`），每个候选创建独立的探测组件，探测成功后返回取消句柄给调用方。所有候选失败后通过 `callback.onError()` 和 `throw RemoteException` 双重通知

到这里，整个大模型调度引擎的 Chat 子系统就讲完了。回顾一下六篇文章的知识脉络：

第一篇《AI 基础设施层宏观设计》——建立全局视角。9 个包的职责划分、三层接口设计（业务层 → 路由层 → 供应层）、配置驱动的设计理念

第二篇《多模型路由与智能选择》——路由入口。`ModelSelector` 如何从配置中选出有序候选列表，优先级排序、首选提升、深度思考过滤、`ModelTarget` 构建

第三篇《三态熔断器与故障转移》——容错骨架。`ModelHealthStore` 三态状态机（`CLOSED` → `OPEN` → `HALF_OPEN`）保护模型健康，`executeWithFallback` 通用同步故障转移执行器

第四篇《Chat 同步调用与模板方法》——协议封装。`AbstractOpenAIStyleChatClient` 模板方法把 `OpenAI` 兼容协议的请求构建、HTTP 调用、响应解析封装为通用骨架，子类通过钩子方法表达供应商差异

第五篇《SSE 流式解析与异步执行》——流式底层。`doStreamChat` 异步提交、`OpenAIStyleSseParser` SSE 解析、`StreamAsyncExecutor` 线程池管理、`StreamCancellationHandle` 取消机制

第六篇《流式路由的首包探测机制》——流式路由。`ProbeStreamBridge` 探测缓冲与首包等待、`probe-and-commit` 模式，在异步回调场景下实现对前端透明的故障转移

六篇文章从宏观到微观、从同步到异步，逐层深入。前三篇搭建路由和容错骨架，第四篇和第五篇深入同步和流式调用的协议层实现，第六篇把所有组件串在一起解决最复杂的流式路由问题。

下一篇离开 Chat 子系统，进入 **Embedding 向量化客户端**——`EmbeddingService / EmbeddingClient` 接口设计、`OllamaEmbeddingClient` 的维度参数处理、`SiliconFlowEmbeddingClient` 的批量分片机制、`RoutingEmbeddingService` 如何复用同一套路由和熔断机制。和 Chat 不同的是，`Embedding` 只有同步调用没有流式——所以路由层直接用 `executeWithFallback` 就够了，不需要首包探测这些复杂机制。